Capítulo 2: Grafos

En este capítulo introduciremos las nociones básicas de teoría de grafos, así como el lenguaje y las propiedades que formarán la base de los temas abordados en los próximos capítulos.

Presentaremos también varios ejemplos, ejercicios e ilustraciones que esperamos permitan al lector adueñarse de este tema.

En cuanto a los algoritmos, en el resto de esta monografía estudiaremos formas de resolver problemas que se plantean sobre las estructuras de grafos que definiremos en este capítulo. Estos problemas tienen gran interés en matemáticas y ciencias de la computación por lo simple que resulta su planteamiento y lo difícil que generalmente es su resolución.

Aparecerán así, de manera natural, las nociones de recursión, iteración, complejidad, etc., que estudiamos en el Capítulo 1.

2.1 Introducción

Un *grafo* es una dupla $G = (V, E)$ compuesta por un conjunto finito $V = V(G)$ de *vértices*, también llamados *nodos* y típicamente dibujados por círculos y un conjunto $E = E(G) \subseteq V_*^2$, que llamaremos conjunto de *aristas*[1], donde hemos definido $V_*^2 = (V \times V) \setminus (\cup_{v \in V}(v, v))$, pues en esta monografía no estamos interesados en estudiar aristas de la forma (v, v), también conocidas como *bucles*.

Un *grafo no-dirigido* (que simplemente llamaremos *grafo*) es una tupla $G = (V, E)$ con las características mencionadas anteriormente, donde una arista $e \in E$ representa una conexión (sin una dirección establecida) entre dos vértices de V y se dibuja por una línea que une estos elementos. Así, la arista $e = (v_1, v_2)$ representa una conexión entre los vértices v_1 y v_2, la cual puede también escribirse como $e = (v_2, v_1)$, en este caso, diremos que los vértices v_1 y v_2 son *adyacentes*. Para simplificar la notación, en esta monografía utilizaremos la convención que la arista e de un grafo no-dirigido aparecerá escrita sólo una vez en la descripción de E y, en caso de existir alguna relación de orden $\leq$ definida sobre el conjunto de vértices V, está será descrita de la forma $e = (v_1, v_2)$ cuando $v_1 \leq v_2$. Sin embargo, para definir correctamente ciertas operaciones entre grafos que veremos en la Sección 2.2.1, las cuales se establecerán mediante operaciones

[1]Hemos adoptado la letra E motivados por el término inglés *edge*, con el cual se llama a este objeto matemático.

usuales entre conjuntos (unión, intersección, complemento, diferencia, etc.), debemos tener en cuenta que, si el par (v_1, v_2) está en E, entonces también lo está el par (v_2, v_1).

Ejemplo 2.1. Sea el grafo $G = (V, E)$, con los vértices $V = \{1, 2, 3, 4, 5, 6\}$ y las aristas $E = \{(1, 2), (1, 3), (2, 4), (2, 6)\}$. Este grafo se representa gráficamente en la Figura 2.1.

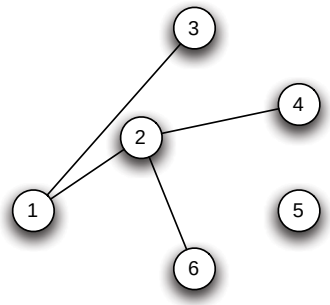


FIGURA 2.1. Ejemplo de un grafo

Por otro lado, un *grafo dirigido* $G = (V, A)$ es un grafo cuyas aristas $a = (v_1, v_2) \in A$ representan una conexión entre los dos vértices v_1 y v_2 en la dirección desde v_1 a v_2, usualmente denotada por una flecha que va desde v_1 hacia v_2. Luego, en un grafo dirigido, una arista nos entrega más información que una arista en un grafo no-dirigido. Dada esta importante diferencia, llamaremos *arco* a un elemento $a = (v_1, v_2) \in A$ y diremos que A es el conjunto de arcos del grafo dirigido G, explicando el cambio de notación de E a A y de e a a. En la Figura 2.2 dibujamos el grafo dirigido $G = (V, A)$, donde sus vértices son $V = \{1, 2, 3, 4, 5, 6\}$ y sus arcos vienen dados por $A = \{(1, 2), (2, 4), (3, 1), (4, 2), (6, 2)\}$.

Resumiendo, la principal diferencia entre grafos dirigidos y no-dirigidos es que, para dos vértices v_1 y v_2 de un grafo dirigido, (v_1, v_2) no representa el mismo arco que (v_2, v_1), pues el primero sólo sirve para "llegar" desde v_1 a v_2, mientras que el segundo tiene el sentido contrario.

Una característica intrínseca a un grafo (no-dirigido y dirigido) es la cardinalidad de los conjuntos de vértices y aristas (o arcos según sea el caso). Estas serán denotadas por $|V|$ y $|E|$ (o $|A|$), respectivamente. En particular, el valor $|E|$ (o $|A|$) es llamado el *tamaño del grafo* G.

Ejercicio 2.1. Mostrar que el tamaño de un grafo no-dirigido G es a lo menos 0 y a lo más $\binom{n}{2}$, siendo n el número de vértices de G. ¿Cuales son las cotas para un grafo dirigido?

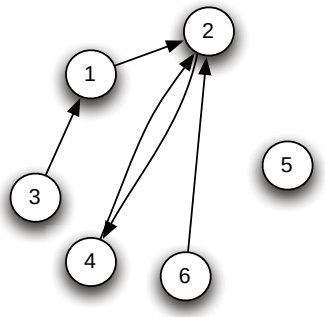


FIGURA 2.2. Ejemplo de un grafo dirigido

2.2 Grafos no-dirigidos

En esta sección a un grafo no-dirigido lo llamaremos simplemente grafo. Un grafo queda completamente definido por sus vértices V y aristas E, aunque su representación gráfica puede variar. Por ejemplo, el grafo de la Figura 2.1, también puede ser representado como en los grafos de la Figura 2.3.

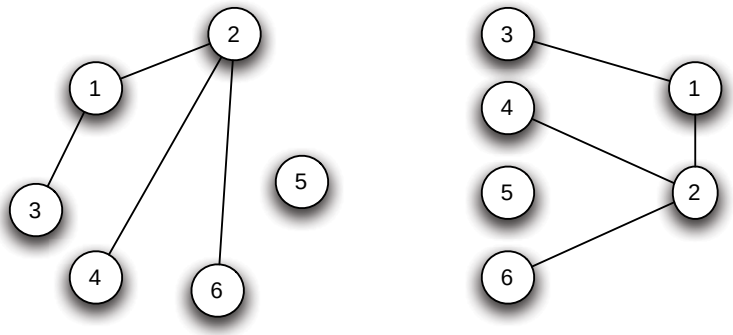


FIGURA 2.3. Otras representaciones gráficas del grafo G

En el contexto de teoría de grafos, en especial en lo referente a grafos no-dirigidos, muchas veces nos enfocamos en la estructura que un grafo pueda tener y a las propiedades que resulten de ésta, quedando relegado a segundo plano el significado que tienen sus vértices y aristas. Es por esto que resulta útil entender cuándo dos grafos tienen propiedades equivalentes aun cuando no sean iguales como objetos matemáticos, ya que los nombres de sus vértices (y, en consecuencia, sus aristas) puedan diferir.

Así, diremos que dos grafos $G = (V, E)$ y $G' = (V', E')$ son *isomorfos* si existen biyecciones, tanto entre sus vértices, como entre sus aristas, es decir, existen dos funciones biyectivas $\varphi_V : V \to V'$ y $\varphi_E : E \to E'$, que satisfacen que $e = (u, v) \in E$ si y sólo si $\varphi_E(e) = (\varphi_V(u), \varphi_V(v)) \in E'$. Por ejemplo, los grafos de la Figura 2.4 son isomorfos, pero distintos.

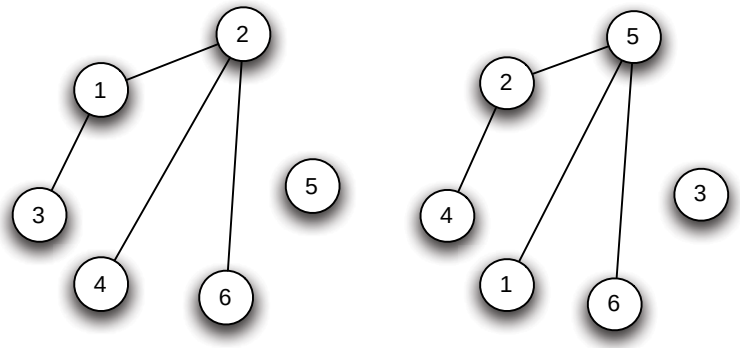


FIGURA 2.4. Grafos isomorfos

En la Sección 2.2.2 estudiaremos algunas estructuras básicas, que se comparten para grafos isomorfos, con algunas de sus propiedades principales.

Una característica muy importante en un grafo no-dirigido es la noción de *arista incidente* y grado de un vértice que definimos a continuación.

Definición 2.1. Diremos que una arista e es *incidente* a un vértice v, si v es uno de los elementos de la arista e. El número de aristas incidentes a un vértice v le llamaremos el *grado* de v y lo denotaremos por $\delta(v)$. En el caso particular que el grado de un vértice v sea 0, diremos que v es un *vértice aislado*.

Para ilustrar esta definición, veamos que en el grafo del Ejemplo 2.1 el grado del vértice 2 es 3, es decir, $\delta(2) = 3$, pues las aristas $(1, 2)$, $(2, 4)$ y $(2, 6)$ inciden en el vértice 2.

Estamos ahora en condiciones de probar nuestro primer resultado en teoría de grafos.

Teorema 2.2. *En todo grafo, el número de vértices de grado impar es par.*

Demostración. Sea $G = (V, E)$ el grafo en consideración. Notemos que cada arista contribuye exactamente 2 veces a la suma total de los grados de los vértices (de hecho podemos calcular el grado de un vértice contando las veces que aparece en el conjunto

E), entonces, sumando los grados de todos los vértices $v \in V$, obtenemos[2]:

$$\sum_{v \in V} \delta(v) = 2|E|. \tag{2.1}$$

Si denotamos por $V' \subseteq V$ el subconjunto de todos los vértices de grado impar, entonces $V'' = V \setminus V'$ es el subconjunto de todos los vértices de grado par y $\sum_{v \in V} \delta(v) = \sum_{v \in V'} \delta(v) + \sum_{v \in V''} \delta(v)$. Obteniendo de (2.1) que

$$\sum_{v \in V'} \delta(v) = 2|E| - \sum_{v \in V''} \delta(v).$$

Pero como el término de la derecha es par, deducimos que la $\sum_{v \in V'} \delta(v)$ es par y, por lo tanto, dado que $\delta(v)$ es impar, para todo $v \in V'$, la única posibilidad para que esto suceda es que el número de vértices de grado impar sea par. □

Observación 2.1. La igualdad (2.1) puede ser reescrita como:

$$\sum_{v \in V} \delta(v) = 0 \mod 2, \tag{2.2}$$

donde la operación $a \mod b$ nos entrega, para dos números enteros a y b, el resto de la división a/b (ver también la Nota 8 al pie de la página 45 del Capítulo 1). Esta última igualdad es llamada el *lema del apretón de manos*[3], dado que expresa el hecho que **¡en cualquier fiesta el número de manos apretadas es par!**.

Ejercicio 2.2. Un grafo cúbico es un grafo de n vértices, donde cada vértice tiene grado 3. Pruebe que si G es un grafo cúbico, entonces $n = |V(G)|$ es par. ¿Cuál es el grafo cúbico de menos vértices?

A lo largo de esta monografía nos resultará útil identificar "partes" de un grafo dado, es decir, un subconjunto de vértices y las aristas que incidan en éstos. Formalmente, diremos que un grafo $G' = (V', E')$ es un *subgrafo* del grafo $G = (V, E)$, si $V' \subseteq V$ y $E' \subseteq E \cap (V' \times V')$. Esta relación la denotaremos por $G' \subseteq G$ y diremos que G *contiene* a G' o que G' *está contenido* en G. Por ejemplo, dos posibles subgrafos del grafo G, dado en el Ejemplo 2.1, son: $G' = (V', E')$, con los vértices $V' = \{1, 2, 4, 6\}$ y las aristas $E' = \{(1, 2), (2, 4), (2, 6)\}$ y $G'' = (V'', E'')$ con $V'' = V'$ y $E' = \{(1, 2), (2, 4)\}$.

Una primera aplicación del estudio de subgrafos es la siguiente definición:

Definición 2.3. Definimos un *camino* como un grafo $P = (V, E)$ donde $V = \{v_0, v_1, \ldots, v_k\}$ y $E = \{(v_0, v_1), (v_1, v_2), \ldots, (v_{k-1}, v_k)\}$. En este caso, diremos que P es un camino que *conecta* o *une* los vértices v_0 y v_k. Definimos también el *largo del camino* de P como el tamaño de este grafo, es decir, el número de aristas k de P. En el caso particular que los extremos del camino P sean el mismo vértice (es decir, $v_0 = v_n$),

[2]En el Ejercicio 2 de la Sección 1.4 de ejercicios se sugiere una forma alternativa para probar la igualdad (2.1).

[3]Se conoce en inglés como *handshaking lemma*.

diremos que el camino es un *ciclo* y lo denotaremos por C. En particular, diremos que P (o C) es un camino (o un ciclo, respectivamente) del grafo G, si además se tiene que $P \subseteq G$ (o $C \subseteq G$, respectivamente).

Observación 2.2. Muchas veces un camino (o un ciclo) de largo k se denota por P_k (o C_k, respectivamente), esta notación será usada en esta monografía sólo cuando no haya posibilidad alguna de confusión.

En el grafo del Ejemplo 2.1 podemos identificar varios caminos posibles, por ejemplo, entre los vértices 3 y 6 existe un único camino que los conecta dado por $P = (\{1, 2, 3, 6\}, \{(1, 2), (1, 3), (2, 6)\})$. Para simplificar la notación, un camino será denotado por la secuencia de vértices o de aristas que lo representan. Así, en el caso del ejemplo anterior, el camino P puede ser descrito simplemente como $(3, 1, 2, 6)$ o $((1, 2), (1, 3), (2, 6))$, respectivamente.

Ejercicio 2.3. Verifique que estas notaciones son coherentes, es decir, que efectivamente describen el mismo camino. Describa, usando los tres tipos de notaciones, el camino que conecta los vértices 3 y 4 en el grafo del Ejemplo 2.1 .

Para establecer propiedades sobre grafos o realizar algoritmos sobre éstos, es muchas veces necesario que los grafos estén "conectados" en algún sentido. En teoría de grafos, esta noción corresponde a la de *grafos conexos*. Usando la Definición 2.3, podemos decir que un grafo es *conexo* si existe un camino entre cada par de vértices distintos del grafo. Por ejemplo, en la Figura 2.5, el grafo de la izquierda es conexo mientras que el grafo de la derecha no lo es, pues no existe un camino que conecte a los vértices 1, 2 ó 3 con los vértices 4 ó 5.

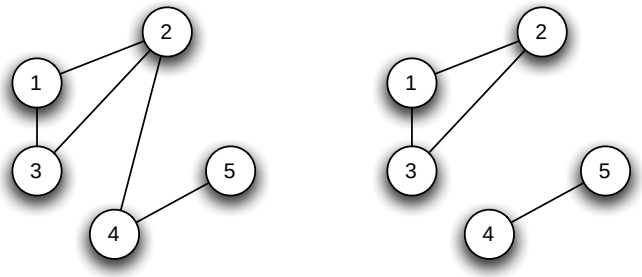


FIGURA 2.5. Ejemplo de un grafo conexo y uno que no lo es

Siempre podemos identificar las *componentes conexas* de un grafo $G = (V, E)$ (que llamaremos simplemente *componentes*) como los subgrafos conexos G_i de G que no están conectados entre ellos y, por lo tanto, sus vértices $V(G_i)$ particionan el conjunto de vértices de G (es decir, $V(G_i) \cap V(G_j) = \emptyset$, para todo $i \neq j$ y $\cup_i V(G_i) = V$) . Por

ejemplo, el grafo de la derecha de la Figura 2.5 tiene dos componentes formadas por los vértices $\{1, 2, 3\}$ y $\{4, 5\}$ y sus respectivas aristas.

Observación 2.3. Notemos que un grafo G compuesto por un solo vértice (i.e. $V(G) = \{v\}$ y $E(G) = \emptyset$) es un grafo conexo, ya que la definición se satisface por vacuidad, pues no existen dos vértices distintos en V.

2.2.1 Operaciones sobre grafos

Vamos a necesitar construir nuevos grafos a partir de algunos ya existentes, para esto, definiremos las siguientes operaciones básicas sobre grafos que nos permitirán sumar o restar vértices y aristas a un grafo $G = (V, E)$ dado.

Si $V' \subset V$ es un subconjunto (estricto) de vértices de G, entonces la *diferencia* entre G y V', denotada por $G \setminus V'$, corresponde al grafo cuyos vértices son $V \setminus V'$ y sus aristas son las aristas E, menos las que inciden en algún vértice de V', es decir, $E(G \setminus V') = E \cap (V \setminus V') \times (V \setminus V')$. Similarmente, si $E' \subset E$ es un subconjunto (estricto) de aristas de G, entonces la *diferencia* entre G y E', denotada por $G \setminus E'$, corresponde al grafo cuyos vértices son los vértices V y cuyas aristas son $E \setminus E'$, es decir, $G \setminus E' = (V, E \setminus E')$. En el caso que estos conjuntos sean un sólo elemento, por ejemplo, $V' = \{v\}$ y $E' = \{e\}$, simplificaremos la notación escribiendo $G \setminus v$ y $G \setminus e$, respectivamente.

En las Figuras 2.6 y 2.7 se ilustra las diferencias entre el grafo del Ejemplo 2.1 y un vértice y de este mismo grafo y un conjunto de aristas, respectivamente.

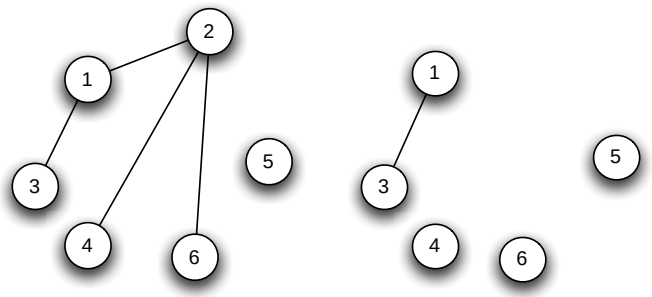


FIGURA 2.6. Diferencia entre un grafo y un conjunto de vértices. Izquierda: G, Derecha: $G \setminus \{2\}$

Para una arista e que une dos vértices *no adyacentes* de un grafo $G = (V, E)$, es decir, $e \in V_*^2 \setminus E$, definimos la *suma* de G y e, denotada por $G + e$, como el grafo obtenido al incluir la arista e en G. En la Figura 2.8 se ilustra la suma del grafo del Ejemplo 2.1 y la arista $(2, 5)$.

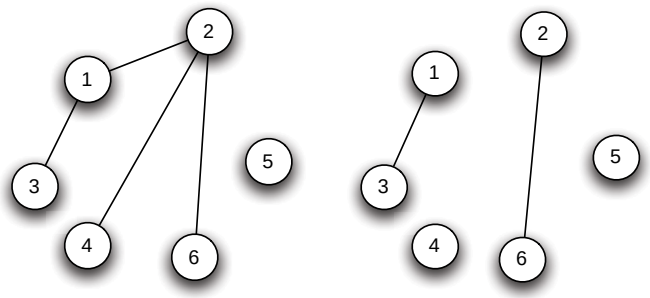


FIGURA 2.7. Diferencia entre un grafo y un conjunto de aristas. Izquierda: G, Derecha: $G \setminus \{(1,2),(2,4)\}$

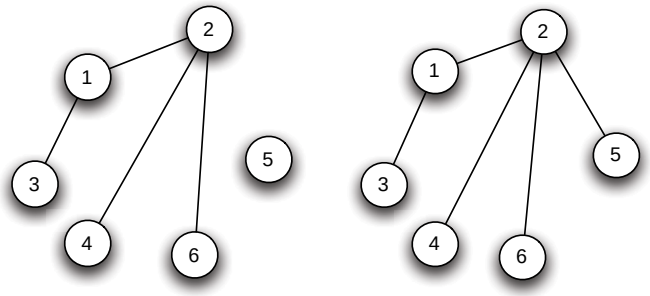


FIGURA 2.8. Suma entre un grafo y una arista. Izquierda: G, Derecha: $G + (2,5)$

Ejercicio 2.4. Sean dos grafos $G = (V, E)$ y $G' = (V', E')$. Deseamos estudiar una posible definición para la operación suma $G + G'$.

(i) Supongamos primero que $V = V'$. Defina $G + G'$ de manera que, si G' tiene sólo una arista, entonces $G + G'$ coincida con la suma entre un grafo y una arista definida anteriormente.

(ii) Usando lo aprendido en el Capítulo 1 construya un algoritmo iterativo que realice tal suma y calcule su complejidad.

(iii) ¿Cómo intentaría definir la suma cuando $V \cap V' = \emptyset$? ¿o cuando $V \cap V' \neq \emptyset$, pero $V \neq V'$?

Finalmente, definimos el *complemento* de un grafo $G = (V, E)$ como el grafo $\bar{G}$ cuyos vértices son los mismos de G, es decir, $V(\bar{G}) = V$ y dos vértices son adyacentes

en G', si y sólo si no lo son en G, es decir, $E(G) = V_*^2 \setminus E$. Se deja como ejercicio al lector calcular y dibujar el complemento del grafo del Ejemplo 2.1. Notemos que el complemento del complemento de un grafo es el grafo original, es decir, $\overline{(\bar{G})} = G$.

Ejercicio 2.5. Probar que, para todo grafo que no sea conexo, se tiene que su complemento necesariamente lo es. Investigar la recíproca, es decir, ver si el complemento de un grafo conexo es conexo o no.

Ejercicio 2.6. Demostrar que para dos grafos G y G', cuyos conjuntos de vértices coinciden, se tiene que: $G \subseteq G'$ si y sólo si $\bar{G}' \subseteq \bar{G}$. ¿Sigue siendo cierta esta propiedad si los vértices de G' están contenidos en los de G?

2.2.2 Algunos grafos particulares

En esta sección presentaremos algunas estructuras particulares de grafos y sub-grafos que nos permitirán plantear los problemas y algoritmos de los próximos capítulos. Estas estructuras se preservan entre grafos isomorfos y tienen también importancia en sí mismas, ya que nos permiten plantear propiedades inherentes a ellas que siguen siendo objeto de investigación en esta área de las matemáticas.

Definición 2.4. Definimos un *árbol* como un grafo conexo que no contiene ciclos.

Un ejemplo de un árbol es el grafo de la Figura 2.9. Planteemos un primer ejercicio asociado a esta estructura:

Ejercicio 2.7. Muestre que en todo árbol siempre hay vértices de grado 1, más aún, la conexidad nos dice que éstos corresponden a los vértices de grado mínimo del árbol. A estos vértices los llamaremos las *hojas* del árbol.

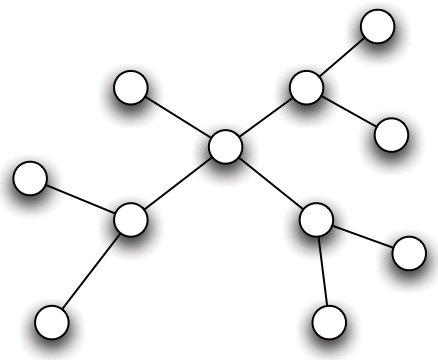


FIGURA 2.9. Ejemplo de un árbol

Una simple caracterización de esta estructura tan particular se plantea a continuación.

Proposición 2.5. *Sea G un grafo conexo con n vértices, entonces G es un árbol si y sólo si G tiene $n-1$ aristas.*

Demostración. Probemos esto por inducción en n. Para $n = 1$ el resultado es directo, pues G es un sólo vértice, que es conexo (ver Observación 2.3) y no contiene ciclos, luego es un árbol y el número de aristas es $n - 1 = 0$. Supongamos ahora que la equivalencia es cierta para grafos de $n - 1$ vértices y sea G un grafo con n vértices, para el cual probaremos la equivalencia establecida en la proposición. En el resto de esta demostración v denotará el vértice de menor grado en G.

Supongamos primero que G es un árbol. Del Ejercicio 2.7 se tiene que v es una hoja, es decir, $\delta(v) = 1$. Si removemos v y su arista incidente, el grafo resultante $G \setminus v$ seguirá siendo un árbol, pues sigue siendo conexo y no contiene ciclos. Aplicando la hipótesis de inducción, obtenemos que $G \setminus v$ tiene $(n-1)-1$ aristas, concluyendo así, que G tenía $n - 1$ aristas.

Recíprocamente, si ahora G tiene $n - 1$ aristas, entonces $\delta(v) = 1$, pues en caso contrario, del lema del apretón de manos (ecuación (2.1)) se deduce que el número de aristas de G sería mayor o igual que n. Luego $G \setminus v$ tiene $(n - 1) - 1$ aristas y, por la hipótesis de inducción, obtenemos que $G \setminus v$ es un árbol. Finalmente, notamos que si volvemos a agregar v y su arista incidente al grafo $G \setminus v$, no se formarán nuevos ciclos (ya que $\delta(v) = 1$), concluyendo que G también es un árbol, lo que prueba la proposición. □

Ejercicio 2.8. Sea $G = (V, E)$ un grafo con n vértices, pruebe que G es un árbol si y sólo si se cumple una de las siguientes caracterizaciones:

1. Cualquier par de vértices de G están conectados exactamente por un camino.
2. Al incluir cualquier arista e que no esté originalmente en G, pero que conecte dos vértices de G (es decir $e \in V_*^2 \setminus E$), el grafo resultante $G + e$ contiene exactamente un ciclo.

Otro tipo de grafo de gran interés es el siguiente:

Definición 2.6. El grafo completo K_n es el grafo de n vértices que contiene todas las aristas posibles, es decir, para cada par de vértices existe una arista que los une.

Ejercicio 2.9. Pruebe que K_n tiene exactamente $\frac{n(n-1)}{2}$ aristas.

El complemento $\bar{K}_n$ de un grafo completo consiste simplemente en n vértices sin conexiones entre sí. Esta estructura tan particular nos permite establecer la siguiente propiedad: K_n está contenido en un grafo G si y sólo si $\bar{K}_n$ está contenido en su complemento $\bar{G}$.

Ejercicio 2.10. Muestre que la aseveración anterior es cierta. ¿Qué sucede si se remplaza K_n por un grafo arbitrario G'? Compare con el Ejercicio 2.6.

Algunos grafos completos particulares son de gran importancia en teoría de grafos, uno de ellos es el *triángulo*, definido como el grafo completo de tres vértices y denotado por K_3. Claramente este grafo también se puede identificar con el ciclo de largo 3, que denotaremos aquí por C_3.

Un hermoso resultado, basado en el lema del apretón de manos (Ecuación (2.1)) y en la *desigualdad de Cauchy-Schwartz*, es el que veremos a continuación.

Teorema 2.7 (Mantel 1907)**.** *Todo grafo con n vértices y tamaño mayor estricto que $\lfloor n^2/4 \rfloor$ contiene un triángulo.*

Demostración. Mostremos la contra-recíproca. Sea $G = (V, E)$ un grafo con n vértices y que no contiene un triángulo, probemos que el tamaño $|E|$ es menor o igual que $\lfloor n^2/4 \rfloor$.

Como G no contiene triángulos, necesariamente se tiene que, para toda arista $e = (v_1, v_2) \in E$, no existen vértices que sean al mismo tiempo adyacentes a v_1 y a v_2. Luego, si denotamos por E_v al conjunto de todas las aristas que inciden en un vértice v, obtenemos que E_{v_1} y E_{v_2} son disjuntos para todo par de vértices distintos v_1 y v_2, lo que en particular implica que los conjuntos E_{v_i}, con $i = 1, ..., n$ (donde hemos denotado por $\{v_1, v_2, ..., v_n\}$ al conjunto de vértices V), son una partición de E, es decir:

$$E = \cup_{i=1}^{n} E_{v_i} \quad \text{y} \quad E_{v_1} \cap E_{v_2} = \emptyset, \forall v_1, v_2 \in V : v_1 \neq v_2. \tag{2.3}$$

Por lo tanto,

$$\delta(v_1) + \delta(v_2) \leq n, \qquad \forall e = (v_1, v_2) \in E. \tag{2.4}$$

Notemos además que, si fijamos un vértice $v_1 \in V$, se tiene que

$$\delta(v_1)^2 = \sum_{e=(v_1,v_2)\in E_{v_1}} \delta(v_1) \leq \sum_{e=(v_1,v_2)\in E_{v_1}} (\delta(v_1) + \delta(v_2)), \tag{2.5}$$

pues en la primera igualdad hemos usado que $\delta(v_1) = \sum_{e=(v_1,v_2)\in E_{v_1}} 1$ y que v_1 está fijo, por lo que podemos pasarlo adentro de la sumatoria y en la desigualdad, que el grado de un vértice es siempre mayor o igual que 0. Entonces, sumando las desigualdades (2.5), para todos los vértices $v_1 \in V$, usando (2.3) y sumando las desigualdades (2.4), para todas las aristas $e = (v_1, v_2) \in E$, respectivamente, obtenemos que

$$\begin{aligned} \sum_{v_1 \in V} \delta(v_1)^2 &\leq \sum_{v_1 \in V} \sum_{e=(v_1,v_2)\in E_{v_1}} (\delta(v_1) + \delta(v_2)) \\ &= \sum_{e=(v_1,v_2)\in E} (\delta(v_1) + \delta(v_2)) \leq n|E|. \end{aligned} \tag{2.6}$$

Por otro lado, del lema del apretón de manos (2.1) y la desigualdad de Cauchy-Schwartz, aplicada a los vectores $(\delta(v_1), \delta(v_2), ..., \delta(v_n))^\top$ y $(1, 1, ..., 1)^\top$ (vector con

n unos), se llega a

$$(2|E|)^2 = \left(\sum_{v_1 \in V} \delta(v_1)\right)^2 \leq n \sum_{v_1 \in V} \delta(v_1)^2.$$

Así, de la ecuación (2.6) se concluye que

$$(2|E|)^2 \leq n^2|E|,$$

es decir, que $|E| \leq n^2/4$. □

Números de Ramsey. Un entretenido juego de ingenio consiste en **probar que en cualquier fiesta de seis personas, siempre hay tres personas que se conocen entre sí o tres personas que no**. Este problema es usualmente llamado el *problema de la fiesta de seis personas*.

Modelaremos este problema usando un grafo G, donde cada vértice representa a una persona y cada arista entre dos vértices significa que las personas representadas, por dichos vértices, se conocen. Con esto, el hecho que tres personas se conozcan entre sí, queda representado por un triángulo K_3 contenido en G.

Dada esta modelación, en el complemento del grafo G, denotado por $\bar{G}$, dos vértices son adyacentes sólo si las personas representadas por dichos vértices no se conocen. Luego, el hecho que tres personas no se conozcan entre sí, queda representado por un triángulo K_3 contenido en $\bar{G}$.

En estos términos, el problema de la fiesta de seis personas puede ser enunciado a través del siguiente teorema. La demostración es dejada como ejercicio.

Teorema 2.8. *Para todo grafo G de seis vértices, se tiene que, G contiene un triángulo, o bien su complemento $\bar{G}$ lo contiene.*

Lo anterior nos permite plantear la siguiente pregunta (conocida como el *problema de la fiesta*): **¿Cuál es la cantidad mínima necesaria de invitados en una fiesta, para que, al menos p se conozcan entre sí, o bien, al menos q no se conozcan entre sí?**

Similarmente al problema de la fiesta de seis personas, el hecho que p personas se conozcan entre sí, queda representado por un grafo completo K_p, contenido en G, y el hecho que q personas no se conozcan entre sí, queda representado por un grafo completo K_q contenido en el complemento $\bar{G}$. Podemos, entonces, establecer la pregunta anterior en términos de grafos como sigue: **¿Cuál es el número natural $r(p,q)$ más pequeño, tal que todo grafo con $r(p,q)$ vértices, contenga a K_p o a $\bar{K}_q$?**[4]

Los valores de $r(p,q)$ son conocidos como los *números de Ramsey*, en honor al matemático inglés Frank P. Ramsey (Cambridge 1903- Londres 1930), quien en sus

[4]Notemos que, gracias al Ejercicio 2.10, que un grafo contenga a $\bar{K}_q$ equivale a decir que su complemento contenga a K_q, lo que para el problema de la fiesta corresponde a que, al menos q no se conozcan entre sí.

sólo 27 años de vida produjo una cantidad extraordinaria de trabajos originales en economía, lógica, filosofía y, por supuesto, matemáticas.

Ejercicio 2.11. Demuestre que $r(p,q) = r(q,p)$, para todo p, $q \in \mathbb{N} \setminus \{0\}$. Muestre además que $r(1,q) = 1$ y $r(2,q) = q$, para todo $q \in \mathbb{N} \setminus \{0\}$. Interprete en términos del problema de la fiesta.

La determinación de los números de Ramsey es un problema abierto en matemáticas y sólo se conocen con exactitud para algunos valores específicos de p y q (ver Tabla 2.1). Sin embargo, una cota superior fácil de calcular, escrita en términos de números combinatoriales, es conocida desde el año 1935 gracias al trabajo de Erdös y Szekeres:

$$r(p,q) \leq \binom{p+q-2}{p-1}. \tag{2.7}$$

p	q	$r(p,q)$
3	3	6
3	4	9
3	5	14
3	6	18
3	7	23
3	8	28
3	9	36
4	4	18
4	5	25

Tabla 2.1. Números de Ramsey conocidos

Otra interesante estructura relacionado con el concepto de subgrafos es la que se presenta a continuación:

Definición 2.9. Un grafo *bipartito* es un grafo $G = (V, E)$ en que el conjunto de vértices se puede particionar en dos subconjuntos V^1 y V^2 (es decir, $V = V^1 \cup V^2$ y $V^1 \cap V^2 = \emptyset$), de manera que las aristas de G tienen un extremo en V^1 y otro en V^2 (es decir, $E \subseteq V^1 \times V^2$).

En la Figura 2.10, el grafo de la izquierda es bipartito, sin embargo, el grafo de la derecha, que corresponde al triángulo K_3, no es bipartito, pues no es posible particionar el conjunto de vértices sin dejar una arista con ambos extremos en el mismo conjunto.

Ejercicio 2.12. Probar que todo subgrafo de un grafo bipartito, es también un grafo bipartito.

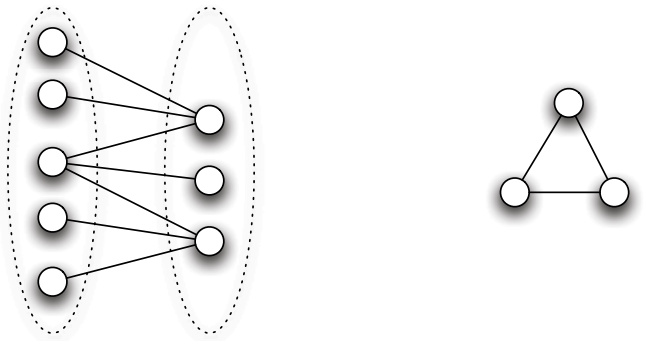


FIGURA 2.10. Ejemplo de un grafo bipartito y un no-bipartito

El ejemplo del lado derecho de la Figura 2.10 podemos generalizarlo a un ciclo de tamaño n. No es difícil ver que la única forma para crear una bipartición en el conjunto de vértices de un ciclo, es dejar a un vértice en V^1 y sus vértices adyacentes en V^2, y así sucesivamente. Esto efectivamente bi-particionará el ciclo si y sólo si n es par.

Ejercicio 2.13. Formalizar la demostración de que un ciclo de tamaño n es bipartito si y sólo si n es par.

Por lo mismo, la existencia de ciclos impares caracteriza a los grafos bipartitos como sigue:

Proposición 2.10. *Un grafo G es bipartito si y sólo si no contiene un ciclo de largo impar.*

Demostración. Debido a los Ejercicios 2.12 y 2.13, el hecho de no contener ciclos impares es claramente una condición necesaria.

Probemos ahora que es también una condición suficiente. Se verifica rápidamente que un grafo es bipartito si y sólo si cada una de sus componentes lo son, luego podemos asumir, sin perdida de generalidad, que el grafo es conexo.

Sea v un vértice de G y definamos V^1 como todos los vértices $v^1 \in V$, tales que, el camino más corto que conecta v y v^1 es de largo par[5]. Definamos también $V^2 = V \backslash V^1$. Notemos que no hay ningúna arista que conecte dos elementos en un mismo conjunto V^i, con $i = 1, 2$, pues esto implicaría la existencia de un ciclo de largo impar. En efecto, denotemos esta arista por $e = (v', v'')$ y llamemos P' y P'' los caminos que

[5]Podemos definir $V^1 = \{v^1 : d(v, v^1) \text{ es par }\}$, donde $d(v, v^1)$ es la distancia entre v y v^1 en un grafo conexo G, la cual, a su vez, se define como el largo del camino más corto que conecta v y v^1 en G, es decir, $d(v, v^1) = \min |P|$; P es un camino que conecta v y v^1. Ver también Ejercicio 4 en la Sección 1.4 de ejercicios.

conectan v con v' y v'', respectivamente. Si $v', v'' \in V^1$, los caminos P' y P'' tienen largo par y entonces el ciclo $C = (P', e, P'')$ tiene largo impar. Ahora, si $v', v'' \in V^2$, entonces los caminos P' y P'' tienen largo impar, pero el ciclo $C = (P', e, P'')$ sigue teniendo largo impar.

Por lo tanto, podemos particionar el conjunto de vértices, usando V^1 y V^2, y lo anterior nos dice que las aristas de G están en $V^1 \times V^2$, concluyendo que G es un grafo bipartito. □

A continuación describimos una estructura de grafos que combina la de grafo completo y bipartito, dadas en las Definiciones 2.6 y 2.9, respectivamente.

Definición 2.11. Diremos que un grafo $G = (V, E)$ es *bipartito-completo*, si éste es bipartito, cuya partición del conjunto de vértices V está dada por V^1 y V^2, y si las aristas de G corresponden a todas las aristas que conectan un vértice de V^1 con uno de V^2 (es decir, $E = V^1 \times V^2$). Si las cardinalidades de V^1 y V^2 son n_1 y n_2, respectivamente, denotaremos a este grafo por K_{n_1,n_2}.

Observación 2.4. El tamaño de un grafo bipartito-completo K_{n_1,n_2} es $n_1 n_2$.

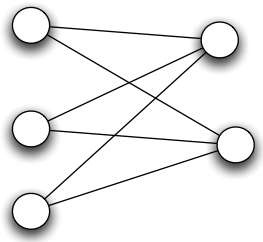


FIGURA 2.11. Ejemplo del grafo bipartito-completo $K_{3,2}$

Ejercicio 2.14. Dibuje los siguientes grafos completos-bipartitos $K_{1,3}$ y $K_{1,n}$. Muestre que los grafos K_{n_1,n_2} y K_{n_2,n_1} son isomorfos.

La introducción de grafos completos-bipartitos nos permite ver que la cota mostrada en el Teorema 2.7 es la mejor posible. En efecto, para todo $n \in \mathbb{N} \setminus \{0\}$ el grafo completo-bipartito $K_{\lfloor n/2 \rfloor, \lceil n/2 \rceil}$ tiene n vértices, tamaño igual a $\lfloor n^2/4 \rfloor$ y no contiene triángulos gracias a la Propiedad 2.10. Mostrando, así, que el Teorema 2.7 de Mantel no se cumple para grafos de tamaño $\lfloor n^2/4 \rfloor$.

Ejercicio 2.15. ¿Cuál de los siguientes grafos es isomorfo a $K_{2,2}$: P_4, C_4 o K_4?

Terminamos esta sección dejando propuesto al lector un entretenido ejercicio que muestra lo simple que es modelar situaciones cotidianas, usando teoría de grafos.

Ejercicio 2.16. El *dilema de los maridos celosos* se trata de tres maridos y sus respectivas esposas que desean cruzar un río. Para aquello cuentan con un sólo bote con capacidad máxima para dos personas. Dado que estos tres maridos son muy celosos, ninguno deja a su mujer en compañía de otro hombre, a menos que él también esté presente. Dibuje un grafo con las distribuciones posibles de las personas y aconseje a estos viajeros cómo cruzar el río.

2.3 Grafos dirigidos

En esta sección adaptaremos algunas definiciones y propiedades establecidas en las secciones anteriores de este capítulo, a un *grafo dirigido*.

Recordemos que un *grafo dirigido* $G = (V, A)$ es una dupla compuesta por un conjunto de vértices $V = V(G)$ y un conjunto de *arcos* $A = A(G) \subseteq V_*^2$, cuyos elementos $a = (v_1, v_2) \in A$ representan una conexión entre los dos vértices v_1 y v_2 en la dirección desde v_1 a v_2. Luego, la principal diferencia entre grafos dirigidos y no-dirigidos es que, para dos vértices v_1 y v_2 del grafo, $a = (v_1, v_2)$ no representa el mismo arco que $a' = (v_2, v_1)$, pues a sólo sirve para "llegar" desde v_1 a v_2, mientras que a' tiene el sentido contrario. Un ejemplo de grafo dirigido fue dado en la Figura 2.2.

Notemos que, si por un instante no consideramos los sentidos en los arcos del grafo de la Figura 2.2, es decir, que sólo nos interesamos en las conexiones establecidas entre los vértices del grafo, obtenemos el grafo no-dirigido introducido en el Ejemplo 2.1 (ver Figura 2.12). Formalizaremos esto en la próxima definición:

Definición 2.12. Sea $G = (V, A)$ un grafo dirigido. La *versión no-dirigida* de (o el *grafo no-dirigido asociado a*) G es el grafo no-dirigido $G' = (V, E)$ con los mismos vértices que G y cuyas aristas conectan dos vértices sólo si existe algún arco en A entre éstos dos vértices, independiente de su dirección. En otras palabras, para dos vértices $v, v' \in V$, $e = (v, v') \in E$ si y sólo si $(v, v') \in A$ o $(v', v) \in A$.

Antes de continuar, explicaremos un ligero cambio de notación con respecto a lo visto en las secciones anteriores. Cuando trabajamos con grafos dirigidos, en general, se necesita establecer claramente la dirección de sus arcos, por lo que en la descripción de un arco, usualmente se explicitan los dos vértices que lo definen y en el orden que lo hacen. Debido a esto, a menudo simplificaremos la notación de los vértices, denotándolos con índices i, j, k, etc., los cuales, a su vez, corresponderán a los subíndices que nos permitan identificar fácilmente a un arco de la forma $a_{ij} = (i, j)$. Esta notación facilitará los cálculos sobre estos arcos y será de gran utilidad en el Capítulo 5 de esta monografía.

A continuación, adaptamos la Definición 2.3 de camino, realizada en el contexto de grafos no-dirigidos, al caso de grafo dirigidos.

Definición 2.13. Sea $G = (V, A)$ un grafo dirigido. Un *camino* entre dos vértices i y j en V es un subgrafo $P = (V', A')$ de G (i.e. $V' \subseteq V$ y $A' \subseteq A$), donde $V' =$

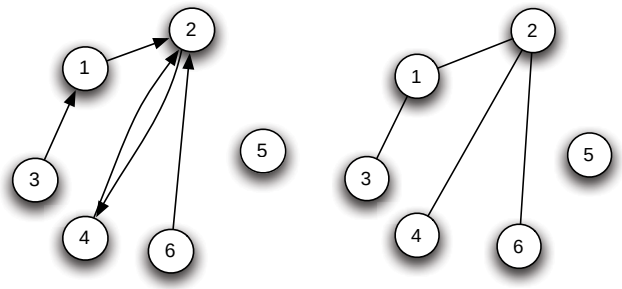


FIGURA 2.12. Ejemplo de un grafo dirigido y su versión no-dirigida

$\{i_0, i_1, \ldots i_l\}$ y $A' = \{(i_0, i_1), (i_1, i_2), \ldots, (i_{l-1}, i_l)\}$, donde $i_0 = i$ e $i_l = j$. Usualmente, el camino P es también identificado como una secuencia de arcos $a_k = (i_k, i_{k+1}) \in A$, con $k = 0, .., l$, o como una secuencia de los vértices $(i_0, i_1, ..., i_{l+1})$, debido a que no hay posibilidad de confusión. Si $i = j$, es decir, si los vértices que se encuentran en los extremos son los mismos, diremos que el camino es un *ciclo*.

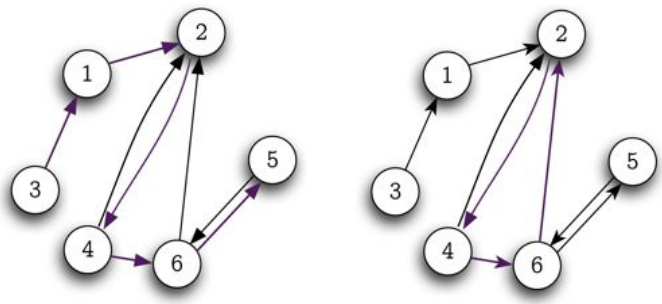


FIGURA 2.13. En púrpura se describe un camino y un ciclo, respectivamente

Una extensión de esta definición, propia de grafos dirigidos, es la siguiente:

Definición 2.14. Sea $G = (V, A)$ un grafo dirigido. Una *cadena* entre dos vértices i y j en V es un camino entre estos vértices en la versión no dirigida de G.

Notemos que si P es una cadena de G y $a = (k, l)$ es uno de los arcos que forman esta cadena, entonces se tiene que $(k, l) \in A$ o bien $(l, k) \in A$. En el primer caso diremos que a es un *arco hacia adelante* y en el segundo caso diremos que a es un *arco hacia atrás* o *en reversa*. Así, una cadena P no es un camino en G si y sólo si existe un arco hacia atrás en P.

Por ejemplo, en el grafo de la Figura 2.13, la secuencia de arcos

$$P = ((3,1),(1,2),(4,2),(4,6),(6,5))$$

es una cadena, pero no es un camino, pues el arco $(4,2)$ es un arco hacia atrás.

Como veremos en la próxima definición, las nociones de grafo conexo, árbol, grafo completo y bipartito se extiende directamente a grafos dirigidos, vía la versión no-dirigida del grafo.

Definición 2.15. Diremos que un grafo dirigido es *conexo (un árbol, completo, bipartito o bipartito-completo)* si su versión no-dirigida es conexa (un árbol, completa, bipartita o bipartita-completa, respectivamente).

Las distintas propiedades asociadas a estas definiciones tienen un interés menor en el caso de grafos dirigidos, por lo que no indagaremos en ellas. Sin embargo, algunas de estas estructuras serán fundamentales para desarrollar algunos razonamientos en los Capítulos 4 y 5.

Las operaciones diferencias, complemento y suma de un arco, se definen de la misma forma que para grafos dirigidos. Se deja propuesto al lector el establecer formalmente estas operaciones.

Cerramos este capítulo, extendiendo la definición de grado de un vértice al contexto de grafos dirigidos.

Definición 2.16. Sea $G = (V, A)$ un grafo dirigido. Para un vértice $v \in V$, el *grado* de v corresponde a la cantidad total de arcos en G que parten o terminan en v (i.e. $(v, v') \in A$ o $(v', v) \in A$), el cual será denotado por $\delta(v)$. Por simplicidad de notación, cuando el vértice sea denotado por un índice i, su grado será denotado simplemente por $\delta_i = \delta(i)$.

En el caso particular que el grado de un vértice v sea 0, diremos que v es un *vértice aislado*.

La principal diferencia entre grafos dirigidos y no-dirigidos con respecto a esta definición, es que, en el caso de grafos dirigidos, podemos contar los arcos que parten y los que salen de un vértice v separadamente, dando origen a los conceptos de *grado saliente* $\delta_+(v) = |\{a = (v, v') \in A\}|$ y *grado entrante* $\delta_-(v) = |\{a = (v', v) \in A\}|$, respectivamente. Obteniendo la relación

$$\delta(v) = \delta_+(v) + \delta_-(v), \quad \forall v \in V.$$

Por ejemplo, en el grafo de la Figura 2.2 notamos que el grado del vértice 2 es igual a $\delta(2) = 4$ y sus grados salientes y entrantes son $\delta_+(2) = 1$ y $\delta_-(2) = 3$.

Ejercicio 2.17. Modele una vecindad del lugar donde vive (por ejemplo, las diez calles más cercanas), usando un grafo dirigido, donde cada vértice represente una intersección entre dos calles y los arcos representen las calles con sus respectivos sentidos. Calcule los grados entrantes y salientes de cada uno de los vértices e identifique si su vecindad tiene alguna estructura particular (conexo, árbol, bipartito, etc.).

2.4 Ejercicios

1. Pruebe que todo grafo (con más de dos vértices) contiene dos vértices de igual grado.
2. Para un grafo $G = (V, E)$, definimos la *matriz de incidencia* A como la matriz de $|V|$ por $|E|$ elementos, cuyas filas están indexadas por los vértices y cuyas columnas están indexadas por las aristas, de tal forma que $A = (A_{ve})$, con $A_{ve} = 1$ si e es incidente en v y $A_{ve} = 0$ si no.
   a) Escriba la matriz de incidencia de todos los grafos que aparecen en este capítulo.
   b) ¿Qué se obtiene al sumar una fila de A? ¿Qué se obtiene al sumar una columna de A?
   c) Muestre el lema del apretón de manos (2.1), usando la matriz de incidencia de un grafo.
   Indicación: Note que puede contar la cantidad de unos que aparecen en A de dos formas distintas: sumando las filas o sumando las columnas.
3. Demuestre que el conjunto de aristas de un grafo $G = (V, E)$ puede ser particionado en ciclos (es decir, existen ciclos C_i, $i = 1, ..., k$ en G tales que $V(C_i) \cap V(C_j) = \emptyset$, para todo $i \neq j$, y $\cup_{i=1}^{k} V(C_i) = V$) si y sólo si cada vértice de G tiene grado par.
4. Definiremos la distancia $d(v, v')$ entre dos vértices v y v' de un grafo conexo G como el largo del camino más corto que conecta v y v' en G, es decir, $d(v, v') = \min |P|$; P es un camino que conecta v y v'. Muestre que d define una *métrica* sobre G.
5. Sea $G = (V, E)$ un grafo de tamaño superior o igual a 2 ($|G| \geq 2$). Mostrar que las siguientes aseveraciones son equivalentes:
   a) Cualquier par de vértices de G forman parte de algún ciclo.
   b) Cualquier par de aristas de G forman parte de algún ciclo.
   c) Para cualquier trío de vértices v_1, v_2 y v_3 de G, existe un camino entre v_1 y v_2 que pasa por v_2.
6. Muestre que para todo grafo conexo G, con al menos dos vértices, existen vértices v_1 y v_2, tales que $G \setminus v_1$ y $G \setminus v_2$ son conexos.
7. Pruebe que un grafo con n vértices y grado mínimo $\delta \geq \frac{n}{2}$ es conexo.
8. Dibuje todos los árboles posibles (salvo isomorfismo) de 4,5 y 9 vértices.
9. Demuestre que un árbol con al menos 2 vértices, contiene al menos 2 vértices de grado 1.
   Indicación: Utilice la igualdad (2.1).
10. Un *árbol generador* T de un grafo G es un árbol que contiene todos los vértices de G (es decir, $V(T) = V(G)$). Muestre que todo grafo conexo contiene un árbol generador.
11. Un grafo que no contiene ciclos lo llamaremos un *bosque*.
    a) Defina un árbol en términos de un bosque.

b) Muestre que un grafo G es un bosque si y sólo si para todo par de vértices distintos v_1 y v_2 de G, existe a lo más un camino entre v_1 y v_2 en G.

12. Demuestre que un bosque de n vértices y k componentes (es decir, k subgrafos conexos que no están conectados entre ellos) tiene $n-k$ aristas. Plantee y estudie la recíproca.
13. La unión entre dos grafos $G = (V, E)$ y $G' = (V', E')$ se define $G \cup G' = (V \cup V', E \cup E')$. Suponiendo que los vértices V y V' son disjuntos y, usando las notaciones dadas en este capítulo, dibuje las uniones de los grafos $G = P_4$ con $G' = C_5$, $G = P_3$ con $G' = K_4$, $G = K_3$ con $G' = K_3$, $G = C_3$ con $G' = K_{1,4}$, y $G = K_3$ con $G' = K_{2,3}$. ¿cómo cambian estos dibujos si ahora asume que los grafos G y G' tienen un vértice en común?.
14. Pruebe formalmente que la unión de dos caminos distintos que conectan los mismos vértices contiene un ciclo.
15. Demuestre que los números de Ramsey satisfacen la siguiente relación de recurrencia (debida a Erdös y Szekeres **[9]**):

$$r(p, q) \leq r(p-1, q) + r(p, q-1).$$

16. Muestre que todo árbol es un grafo bipartito. ¿qué árboles son grafos bipartitos-completos?
17. Pruebe que si G es un grafo con n vértices y $\lfloor n^2/4 \rfloor$ aristas, que no contiene triángulos, entonces G es isomorfo al grafo bipartito-completo $K_{\lfloor n/2 \rfloor, \lceil n/2 \rceil}$.
18. Sea v un vértice arbitrario de un grafo conexo G. Usando la definición de distancia entre dos vértices $d(v, v')$ dada en el Ejercicio 4, muestre que G es bipartito si y sólo si $d(v, v') \neq d(v, v'')$, para toda arista (v', v'') de G.
19. Considere un grafo dirigido $G = (V, A)$ con la propiedad adicional que para cada par de vértices $i, j \in V$ uno y sólo uno de los arcos (i, j) o (j, i) existe. Pruebe que G contiene un camino P que recorre todos los vértices de G sólo una vez.
20. Sea $G = (V, E)$ un grafo no-dirigido conexo con una cantidad par de vértices. Demuestre que siempre se puede asignar una dirección a los vértices de G (obteniendo así un grafo dirigido $G' = (V, A)$, donde cada par de vértices son adyacentes por, a lo más, un arco) de manera que el grado saliente $\delta_+(v)$ en G' sea par, para todo vértice $v \in V$.

Capítulo 3: Árbol recubridor de costo mínimo

3.1 Árbol recubridor de costo mínimo

Suponga que usted está encargado de construir los caminos que unirán a un conjunto de pueblos que hasta ahora no tienen contacto entre sí. Se estudiaron las posibles conexiones entre los pueblos y el costo asociado a construir estos caminos, obteniendo la situación de la Figura 3.1.

Usted debe decidir cuál de estos caminos construir, de forma que todas las ciudades queden conectadas y, dado que sus recursos son escasos, que el costo total de construir estos caminos sea el menor posible. ¿Cómo encontrar la solución de este problema?

Pensemos ahora la situación como si fuera un grafo $G = (V, E)$ donde, los pueblos son los vértices V y los posibles caminos son las aristas E. Además, se tiene un costo c_e asociado a cada arista e, que asumiremos que es positivo ($c_e \geq 0 \ \forall e \in E$).

Lo que tenemos que decidir, es qué conjunto de caminos (subconjunto E' de aristas del grafo) construir, de forma que todas las ciudades estén conectadas (que el grafo (V, E') sea conexo) y que el "costo" total ($\sum_{e \in E'} c_e$) sea el menor posible.

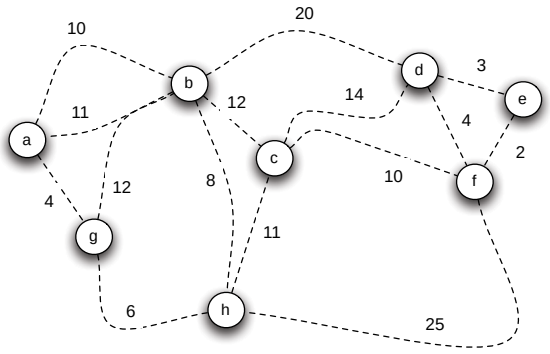


FIGURA 3.1. Grafo de los posibles caminos entre los pueblos. El número al lado de cada arista corresponde al costo de construcción de ese camino

Algunas cosas están claras: en la solución no deberían aparecer ciclos, pues si los hubieran, entonces al sacar una arista del ciclo, se obtiene una solución con menos aristas (o sea, menos costosa) y el grafo seguiría siendo conexo. O sea, lo que necesitamos es un subgrafo que:

1. incluya a todos los vértices del grafo original
2. sea conexo
3. no tenga ciclos

Recordemos del Capítulo 2 que un grafo conexo sin ciclos es un árbol. En nuestro caso, necesitamos un árbol que contenga todos los vértices de V, este tipo de árbol se conoce como *árbol recubridor* de G.

En general, hay muchos árboles recubridores para un grafo dado, pero en nuestro caso, buscamos el árbol recubridor cuyo "costo" (o sea, la suma de los costos de sus aristas) sea el menor posible.

Problema 3.1 (Árbol recubridor de costo mínimo)**.** *Dado un grafo $G = (V, E)$ y un costo c_e, para cada arista $e \in E$, encontrar un árbol recubridor de G de costo mínimo.*

En la Figura 3.2, se pueden ver dos posibles soluciones a este problema: ¿Habrá una mejor? ¿cómo resolvemos este problema? ¿qué tipo de algoritmo utilizamos para resolver este problema?

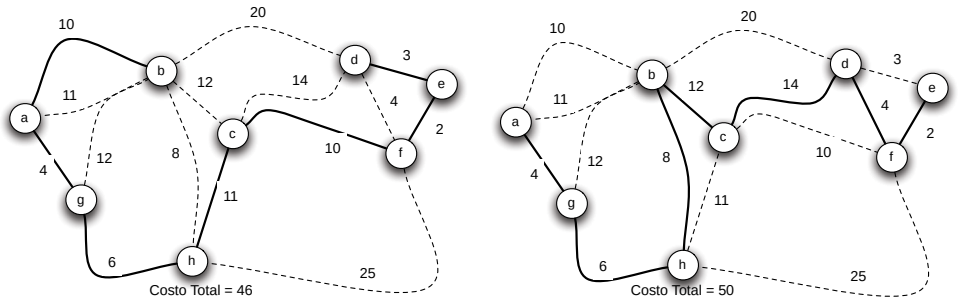


FIGURA 3.2. Dos soluciones al problema anterior

Nuestra intuición puede ayudarnos: si dos pueblos se unen por un camino con muy bajo costo, posiblemente voy a construir ese camino. Este sencillo razonamiento lo podemos llevar a un algoritmo del tipo glotón: ir agregando una a una las aristas de menor costo, mientras el subgrafo sea un árbol. Este algoritmo es conocido como el algoritmo de Prim, en honor a Robert C. Prim quien lo descubrió en 1957.

Solución 3.1 (Algoritmo de Prim)**.** *Sea $T = \emptyset$ un conjunto de aristas, agregar a T la arista en $E \setminus T$ de menor costo, tal que $H = (V, T)$ no tenga ciclos. Detenerse cuando H sea un árbol recubridor.*

En la Figura 3.3 se muestra la solución de nuestro problema inicial, cuyo costo total es 44.

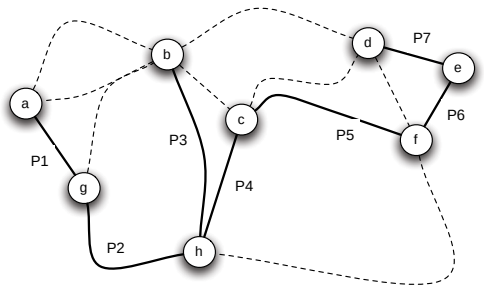


FIGURA 3.3. Pasos del Algoritmo de Prim

Escribamos nuevamente el algoritmo de forma más algorítmica para poder analizar su correctitud y calcular su complejidad.

Algoritmo 3.1 Algoritmo de Prim

Entrada: $G = (V, E)$: grafo
1: $T \leftarrow \emptyset$ /* conjunto de aristas */
2: $u \leftarrow$ vértice inicial cualquiera
3: $U \leftarrow \{u\}$ /* conjunto de vértices */
4: **mientras** $U \neq V$
5: $(u, v) \leftarrow$ arista de menor costo, tal que $u \in U$ y $v \in V \setminus U$.
6: $T = T \bigcup \{(u, v)\}$
7: $U = U \bigcup \{v\}$
8: **fin**
Salida: T árbol recubridor de costo mínimo.

Probemos que el algoritmo de Prim encuentra un árbol recubridor de costo mínimo:

Demostración. Probaremos por inducción en el tamaño de U que en cada paso el subgrafo (U, T) es un sub-árbol de un árbol recubridor de costo mínimo.

El caso base es trivialmente cierto: si el grafo sólo tiene un vértice $\{u\}$, entonces $(\{u\}, \emptyset)$ es un sub-árbol del árbol recubridor de costo mínimo. Supongamos ahora que es cierto para (U, T) y llamemos $U' = U \bigcup \{v\}$ y $T' = T \bigcup \{(u, v)\}$ para el arco (u, v) del paso 5. Podemos observar que (U', T') es conexo y sigue siendo un sub-árbol, ya que se agregó una arista y un vértice, es decir, $|T'| = |U'| - 1$. Sólo falta demostrar que es un sub-árbol de un árbol recubridor de costo mínimo.

Por hipótesis de inducción, T es un sub-árbol de un árbol recubridor de costo mínimo T_M. Si suponemos que la arista $e = (u, v)$ no está en T_M, entonces $T_M \cup e$

posee un ciclo. Luego, hay otra arista e' de ese ciclo que conecta U con $V \setminus U$. Por la definición de la arista e (línea 5), el costo de e' tiene que ser mayor o igual al costo de e, por lo tanto, el árbol recubridor $T_M \cup \{e\} \setminus \{e'\}$ tiene costo menor o igual que T_M y este árbol contiene a T' como sub-árbol, probando el resultado. □

Estudiemos ahora la complejidad del algoritmo de Prim en función de la cantidad de vértices $|V|$. El ciclo **mientras** (líneas 4-8) se repite $|V| - 1$ veces y cada iteración de este ciclo toma $\mathcal{O}(|V|)$, pues debe encontrar la arista de la línea 5, por lo que la complejidad de este algoritmo es $\mathcal{O}(|V|^2)$.

Teorema 3.1. *El algoritmo de Prim corre en tiempo* $\mathcal{O}(|V|^2)$.

Existe otro algoritmo glotón para encontrar un árbol recubridor de costo mínimo, que en ciertos casos puede ser más eficiente que el algoritmo de Prim.

La idea es muy parecida: en cada iteración, agregar la arista de menor costo (¡aunque no quede conectada!) mientras no se forme un ciclo. Este algoritmo se conoce como el Algoritmo de Kruskal, en honor a Joseph Kruskal, quien lo publicó en el año 1956.

Algoritmo 3.2 Algoritmo de Kruskal

Entrada: $G = (V, E)$: grafo
1: $T \leftarrow \emptyset$ /* conjunto de aristas */
2: $u \leftarrow$ vértice inicial cualquiera
3: $U \leftarrow \{u\}$ /* conjunto de vértices */
4: **mientras** $U \neq V$
5: $(u, v) \leftarrow$ arista de menor costo, tal que $T \bigcup \{(u, v)$ no tiene ciclos.
6: $T = T \bigcup \{(u, v)\}$
7: $U = U \bigcup \{v\}$
8: **fin**
Salida: T árbol recubridor de costo mínimo.

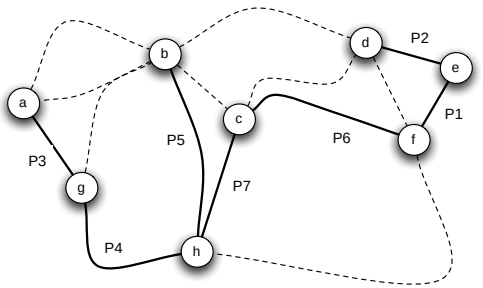


FIGURA 3.4. Pasos del Algoritmo de Kruskal (comparar con Figura 3.3)

Podemos ver que la única diferencia entre el algoritmo de Prim y el algoritmo de Kruskal está en la línea 5 del algoritmo. Dejamos como ejercicio para el lector demostrar que el algoritmo de Kruskal termina con un árbol recubridor de costo mínimo.

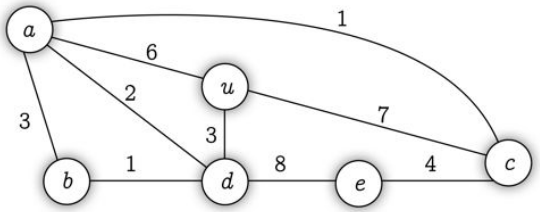
Podemos implementar el algoritmo de Kruskal de una forma más eficiente: antes de ejecutar el algoritmo, ordenamos las aristas del grafo por su costo en forma creciente, lo que requiere un tiempo $\mathcal{O}(|E|\log|E|)$ (ver Capítulo 1). Teniendo las aristas ordenadas, el paso de la línea 5 se puede realizar en forma más eficiente, obteniendo que la complejidad del algoritmo de Kruskal es $\mathcal{O}(|E|\log|E|)$.

Teorema 3.2. *El algoritmo de Kruskal corre en tiempo* $\mathcal{O}(|E|\log|E|)$.

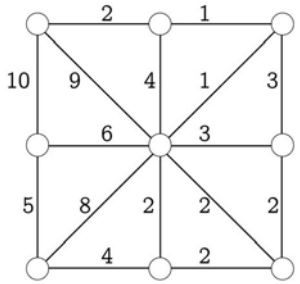
¿Cuál algoritmo conviene usar? Eso depende del número de aristas. Si casi todas las aristas posibles están en el grafo (es decir, $|E| = \mathcal{O}(|V|^2)$), la complejidad del algoritmo de Kruskal sería $\mathcal{O}(|V|^2\log|V|)$ y, por lo tanto, el algoritmo de Prim sería más eficiente. Por otro lado, si el número de aristas es $|E| = \mathcal{O}(|V|)$, el algoritmo de Kruskal correría en tiempo $\mathcal{O}(|V|\log|V|)$ que es menor que el tiempo necesario por el algoritmo de Prim. En nuestro problema inicial, podemos ver que el número de aristas es bastante bajo, pues a una ciudad le interesa conectarse con sus vecinos más cercanos, pero no con aquellos a los que puedo acceder pasando por otra ciudad antes. Por esto, es posible que el algoritmo de Kruskal sea el más adecuado para resolver este tipo de problemas.

3.2 Ejercicios

1. ¿Cuántos árboles recubridores tienen los grafos completos K_3, K_4 y K_5?
2. En el siguiente grafo, encuentre el árbol recubridor de costo mínimo, usando los algoritmos de Prim y de Kruskal.



3. En el siguiente grafo, encuentre el árbol recubridor de costo mínimo, usando los algoritmos de Prim y de Kruskal.



4. En el grafo de la figura anterior, encuentre dos árboles recubridores de peso mínimo distintos.
5. Supongamos que ahora deseamos encontrar el árbol recubridor de peso máximo. ¿Cómo resolver este problema?
6. ¿Cuántos árboles recubridores tiene el grafo completo K_n?
7. Se desea construir un acueducto que una las ciudades de Mafil, Valdivia, Corral, Paillaco y Los Lagos. El costo en miles de millones de pesos de cada tramo viene dado en la siguiente tabla:

	Mafil	Valdivia	Corral	Los Lagos
Valdivia	7			
Corral	19	6		
Los Lagos	5	17	9	
Paillaco	13	5	7	14

¿Qué tramos deberían construirse para gastar la menor cantidad posible de dinero?

Capítulo 4: Camino más corto

4.1 Camino más corto con costos positivos

Supongamos que estamos a cargo de una pizzería a domicilio y, como es usual, nos comprometemos a entregar el pedido en menos de 30 minutos. Para esto, debemos evaluar si es efectivamente posible llegar desde el local a todos los lugares de despacho en menos de ese tiempo. El problema es complicado, hay muchos caminos posible entre la pizzería y cada lugar de despacho, hay calles que son más lentas que otras y, además tiene que preocuparse de respetar el sentido de las calles. ¿Podemos llegar a cada lugar de despacho en menos de 30 minutos? ¿Cuál es el camino más corto entre la pizzería y un domicilio dado?

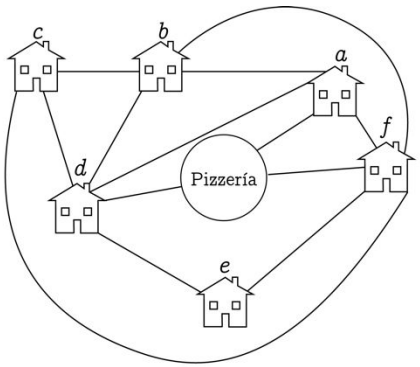


FIGURA 4.1. Ejemplo de una ciudad

Para modelar nuestro problema, representaremos nuestra ciudad por un grafo $G = (V, E)$ donde cada arista es una calle de la ciudad, donde cada arista $e \in E$ tiene un costo asociado c_e (por ejemplo, el tiempo que demora atravesar la calle e), cuyo valor es positivo.

Podemos incluso modelar el sentido de las calles usando grafos dirigidos, la dirección del arco corresponde al sentido de la calle (si la calle es doble sentido, podemos agregar dos arcos, uno en cada sentido de la calle). En este capítulo estudiaremos el caso de grafo no-dirigido, pero todos los resultados pueden extenderse al caso dirigido fácilmente.

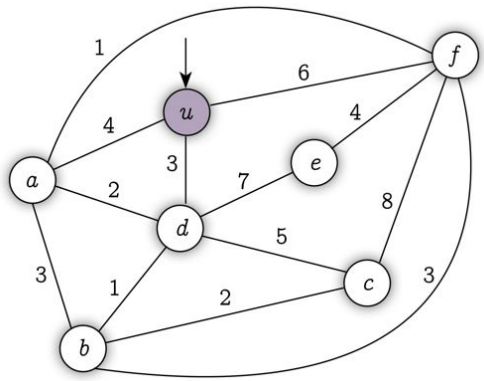


FIGURA 4.2. Grafo asociado a la ciudad anterior

Intentemos, por ejemplo, resolver este problema en el grafo de la Figura 4.2. ¿Cuánto nos demoramos desde la pizzería (vértice u) a un vértice cualquiera? Como vemos, incluso para un grafo tan simple como éste, hay muchos caminos de un vértice a otro, por lo que la estrategia de analizar todos los caminos posibles y escoger el más corto no es un buen algoritmo (o al menos, no es "eficiente").

Sin embargo, para salir de la pizzería, debemos salir por alguno de los tres vértices vecinos a ella (de costo 3, 4 y 6), por lo tanto, podemos asegurar que a donde vayamos, nos demoraremos al menos 3 minutos. Más aún, podemos estar seguros de que el camino más corto de u a d es de largo 3 (pues cualquier otro camino, tomará una distancia mayor que 4 o 6, dependiendo de si salimos por el vértice a o f, respectivamente). Recordemos que esto lo podemos saber gracias a que el costo de las aristas no es negativo.

¿Podemos repetir la idea anterior? Por ejemplo, digamos que queremos ir a otro vértice cualquiera (y recordemos que ya sabemos llegar de u a d). Entonces, el camino más corto de u a ese vértice, o bien, visita d y sale por un vecino de d, o sale por un vecino de u. Es decir, el camino visitará los vértices "conocidos" (vértices grises en la figura) y saldrá por algún vecino de éstos. Así, sabemos que el camino más corto de u a cualquier otro vértice, tiene largo al menos 4, que es el menor costo entre los caminos de u a un vecino no conocido, en este caso, u a d y luego b.

Denotemos por T el conjunto de vértices "conocidos" (vértices grises en la figura) y almacenemos en un vector L las distancias de nuestro vértice inicial u a cada vértice, usando sólo vértices grises. Entonces, el algoritmo que acabamos de hacer consiste en agregar el vértice de no-conocido, cuyo camino tenga el menor costo (en este caso b), verificamos los vecinos de v y actualizamos el valor del vector L para esos vértices, si es que la distancia a través del vértice v es menor. Por ejemplo, en la Figura 4.3,

agregamos el vértice b a T y actualizamos el costo de llegar a c de 8 a 6, ya que ahora este vértice está a distancia 2 de los vértices en T.

El algoritmo que acabamos de explicar, podemos aplicarlo recursivamente hasta incluir todos los vértices del grafo en T. Este algoritmo se conoce como "algoritmo de Dijkstra", en honor al matemático holandés Egsger Dijkstra quien lo describió en 1959.

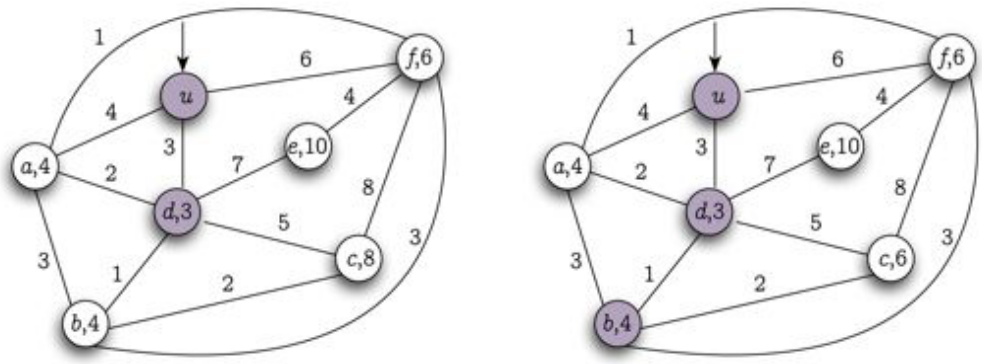


FIGURA 4.3. Grafo asociado a la ciudad anterior

Algoritmo 4.1 Algoritmo de Dijkstra

Entrada: $G = (V, E)$: grafo dirigido, $c : E \to \mathbb{R}^+$ costo de cada arista, u vértice inicial.

1: $L[u] \leftarrow 0$
2: $L[v] \leftarrow c(u, v)$, para todo v vecino de u.
3: $L[v] \leftarrow \infty$, para los vértices v restantes.
4: $T \leftarrow \{u\}$ /* conjunto de vértices que sabemos llegar */
5: **mientras** $T \neq V$
6: w vértice en $V \setminus T$ con el menor valor $L[w]$.
7: $T \leftarrow T \bigcup \{w\}$
8: **para** v vecino de T
9: **si** $L[v] > L[w] + c(v, w)$ **entonces**
10: $L[v] = L[w] + c(v, w)$
11: **fin**
12: **fin**
13: **fin**

Salida: $L[v]$ distancia más corta de u al vértice v $\forall v \in V$.

En la Figura 4.4 podemos ver la iteración final del Algoritmo de Dijkstra al aplicárselo a nuestro problema inicial. Notemos que, si bien sólo tenemos las distancias $L[v]$ desde el vértice u a cada vértice, es fácil reconstruir el camino que debemos utilizar para llegar de u a v en forma óptima. Por ejemplo, el camino más corto de u a e tiene costo 9, por lo que necesariamente tiene que llegar a él a través de f y así sucesivamente, hasta reconstruir el camino $u - a - f - e$.

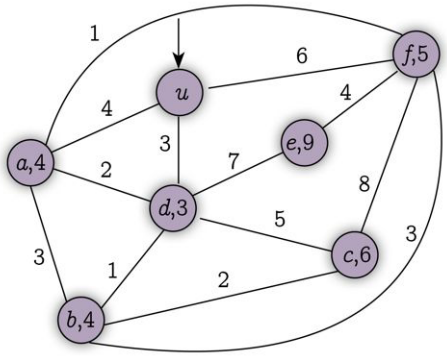


FIGURA 4.4. Resultado del Algoritmo de Dijkstra

También podemos ver en el ejemplo que, cuando un vértice v entra al conjunto T, el valor de $L[v]$ no varía en los siguientes pasos. Esta propiedad es la que demostremos para verificar que el algoritmo efectivamente calcula la distancia más corta desde u al resto de los vértices de G.

Demostración. Demostraremos que en cada paso se cumplen dos condiciones:

1. Para todo vértice $v \in T$, el valor de $L[v]$ es igual al largo del camino más corto desde u a v
2. Para todo vértice $v \notin T$, el valor de $L[v]$ es igual al largo del camino más corto desde u a v, pasando solamente por los vértices de T.

Estas propiedades las demostraremos por inducción en el tamaño de T. Podemos ver que si estas propiedades son verdaderas, en particular cuando T contenga todos los vértices del grafo V, significará que $L[v]$ son los valores que buscamos.

Probemos las propiedades anteriores. En el caso base ($|T| = 1$, es decir, $T = \{u\}$), se cumple gracias a las primeras tres líneas del algoritmo, donde se asignan los valores de L de forma de cumplir con las condiciones.

Probemos ahora la inducción. Supongamos que estamos en el paso 7 del algoritmo y vamos a agregar w a T. Por hipótesis de inducción, los vértices de T tienen el valor correcto de L, por lo que, para cumplir la primera condición sólo falta probar que $L[w]$ tiene el valor correcto. Supongamos que no se cumple, es decir, existe un camino P desde u a w de costo $K < L[v]$ (ver Figura 4.5). Por hipótesis de inducción, este

camino tiene que utilizar otros vértices que no estén en T. Sea z el primer vértice del camino P que no está en T. Por hipótesis de inducción, el costo del camino en P, desde u a z es $L[z]$ y por ser z un vértice intermedio en el camino más corto desde u a w, se debe cumplir que $L[z] < K$, luego, se concluye que $L[z] < L[v]$. Pero por otro lado, en la línea 6 elegimos a w como aquel vértice con menor valor en L que todos los otros vértices fuera de T, en particular el vértice z, por lo tanto, $L[w] \leq L[z]$, lo que nos lleva a una contradicción. Esto demuestra la primera condición enunciada al principio de la demostración. La segunda condición, se cumple gracias a las reasignaciones de la línea 10. □

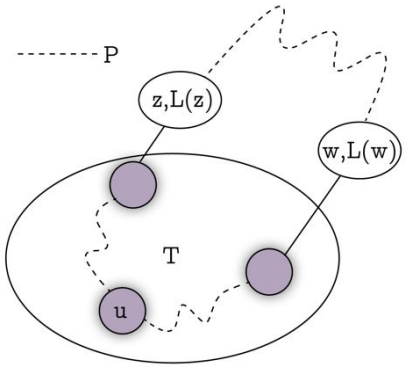


FIGURA 4.5. Demostración del algoritmo de Dijkstra

Estudiemos la complejidad del algoritmo de Dijkstra. El ciclo **mientras** (líneas 5 - 13) se repite $|V|-1$ veces y cada repetición del ciclo requiere encontrar el vértice fuera de T de menor distancia (línea 6) que se puede realizar en $\mathcal{O}(|V|)$ y la modificación de los vecinos (línea 10) que se realiza a lo más $|V|$ veces en cada iteración. Por lo tanto, la complejidad del algoritmo de Dijkstra es $\mathcal{O}(|V|^2)$.

Teorema 4.1. *El Algoritmo de Dijkstra corre en tiempo* $\mathcal{O}(|V|^2)$.

4.2 Camino más corto con costos negativos

El algoritmo de Dijkstra tiene un problema: los costos de las aristas tienen que ser positivos. Esto es un problema, pues en muchas aplicaciones los costos pueden ser negativos. Por ejemplo, pensemos que los caminos son canales con agua, por lo que el "costo" (por ejemplo, de energía) al atravesar una arista puede ser positivo (en contra de la corriente) o negativo (a favor de la corriente).

En el ejemplo de la Figura 4.6 hemos cambiado el costo de la arista c con f por -4. Podemos ver que la lógica del algoritmo de Dijkstra ya no es válida, ya que si

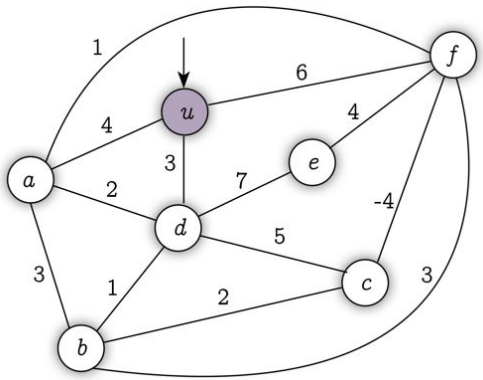


FIGURA 4.6. Ejemplo del problema

bien llegar u a f tiene largo 6, esto nos permitiría llegar a c con un camino de largo $6 - 4 = 2$, por lo que la demostración del teorema ya no es cierta.

Más aún, se puede dar el caso que el grafo contenga un ciclo, cuya suma de costos sea negativo. Esto es complicado, pues el concepto de camino más corto pierde sentido, ya que el camino más corto tiene costo $-\infty$, puesto que para cualquier camino de un vértice a otro, existe otro camino de costo menor, simplemente pasando suficientes veces por el ciclo de costo negativo.

Por todo esto, necesitamos un nuevo algoritmo que sea capaz de encontrar el camino más corto, aun si hay costos negativos, y si aparecen ciclos de costo negativos, darse cuenta de esto e indicarlos. El principal algoritmo para esto es conocido como Algoritmo de Ford-Bellman, desarrollado por Richard Bellman y Lester Ford.

La idea es similar a la de Dijkstra, almacenaremos en un vector L la distancia desde u al vértice, sin embargo, modificaremos la forma de actualizar este valor. En vez de ir agregando glotonamente vértices como lo hace Dijkstra, analizaremos cada arista del grafo para chequear si nos acerca o no a otros vértices. Repitiendo este procedimiento $|V| - 1$ veces, podemos estar seguro que el valor de L fue calculado correctamente.

En la Figura 4.7 podemos ver el resultado de ejecutar el Algoritmo de Ford-Bellman sobre el ejemplo de la Figura 4.6.

Adicionalmente, el algoritmo nos permite detectar ciclos negativos: si al terminar el algoritmo hay una arista (v, w) tal que $L[w] > L[v] + c(v, w)$ significa que la arista está en un ciclo de costo negativo, por lo que el problema del camino más corto no tiene sentido.

Probemos que el Algoritmo de Ford-Bellman efectivamente encuentra el camino más corto desde u a todos los vértices.

Algoritmo 4.2 Algoritmo de Ford-Bellman

Entrada: $G = (V, E)$: grafo dirigido, $c : E \to \mathbb{R}$ costo de cada arista, u vértice inicial.

1: $L[u] \leftarrow 0$
2: $L[v] \leftarrow \infty$, para todo $v \neq u$.
3: **para** $i = 1 \ldots |V| - 1$
4: **para** cada arista $(v, w) \in E$
5: **si** $L[w] > L[v] + c(v, w)$ **entonces**
6: $L[w] \leftarrow L[v] + c(v, w)$
7: **fin**
8: **fin**
9: **fin**

Salida: $L[v]$ distancia más corta de u al vértice v $\forall v \in V$.

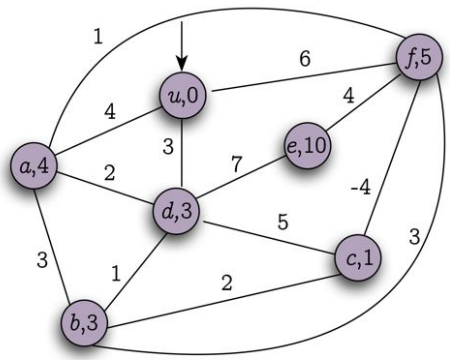


FIGURA 4.7. Resultado del algoritmo de Ford-Bellman

Demostración. Probaremos por inducción en el número de repeticiones del ciclo **para** (líneas 3-9) que se cumplen las siguientes propiedades:

1. Si $L[v] \neq \infty$, entonces $L[v]$ es igual al largo de algún camino de u a v.
2. Si hay un camino de u a v con a lo más i vértices, entonces $L[v]$ es a lo más el costo del camino más corto de u a v con a lo más i vértices.

En el caso base $i = 0$, las condiciones iniciales $L[u] = 0$ y $L[v] = \infty$ $\forall v \neq u$ aseguran que esto se cumpla.

Realicemos ahora la inducción. Para la primera propiedad, supongamos que estamos actualizando el costo de $L[w]$ por $L[v] + c(v, w)$. Por hipótesis de inducción, $L[v]$ es el costo de un camino de u a v, luego, $L[w]$ es el costo del camino que va de u a v y luego a w, por lo que la primera propiedad es cierta.

Probemos la segunda propiedad. Tomemos el camino más corto desde u a v con a lo más i aristas. Sea z el último vértice visitado por el camino, antes de llegar a v. Entonces, necesariamente este camino de u a z es el camino más corto, usando a lo más $i-1$ aristas. Por hipótesis de inducción, el valor de $L[z]$ es a lo más el costo de este camino de u a z, por lo que $L[z] + c(z, v)$ es a lo más el costo del camino más corto de u a v, usando a lo más i aristas, por lo que al final del i-ésimo ciclo **para**, el valor de $L[z]$ cumplirá la segunda condición.

Para terminar la demostración, notemos que cuando $i = |V|$, las propiedades nos aseguran que el valor de $L[v]$ es el costo del camino más corto de u a v, usando a lo más $|V|$ vértices, que es el valor que buscamos (a menos que existen ciclos negativos). □

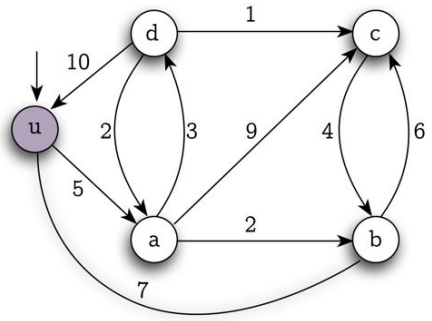
Estudiemos la eficiencia del Algoritmo de Ford-Bellman. El algoritmo incluye dos ciclos **para** anidados: el primero se repite $|V| - 1$ veces y el segundo, $|E|$ veces, por lo que la complejidad del algoritmo es $\mathcal{O}(|V| \cdot |E|)$.

Teorema 4.2. *El Algoritmo de Ford-Bellman corre en tiempo* $\mathcal{O}(|V| \cdot |E|)$.

Recordemos que el algoritmo de Dijkstra corre en tiempo $\mathcal{O}(|V|^2)$, es decir, es más eficiente. Debido a esto, para resolver el problema del camino más corto, comúnmente se utiliza el algoritmo de Dijkstra, a menos de que existan costos no-negativos en las aristas, en cuyo caso usamos Ford-Bellman.

4.3 Ejercicios

1. Encontrar los caminos más cortos para el grafo de la siguiente figura



2. En una empresa se instala una red de N computadores, todos conectados entre sí y además conectados a un supercomputador central (SC). Para testear la velocidad de la comunicación en la red, los técnicos deciden probar la eficiencia del sistema enviando N-1 mensajes en forma paralela (el SC tiene N procesadores), desde el SC hacia los demás equipos. Desafortunadamente, el SC está conectado solamente a uno de los equipos. Describa de qué manera se podría realizar el envío de mensajes, de tal forma que se minimice el tiempo entre el envío por parte del SC y la recepción por parte de un equipo (considere que el tiempo de demora del mensaje es $a_{i,j}$ entre el equipo i y j. El SC es el equipo 1).
3. Encuentre un ejemplo de un grafo con aristas de costo negativo (pero sin ciclos de costo negativo), donde Dijkstra entregue una solución incorrecta.
4. Suponga que tenemos aristas con costo negativos. Una solución sería sumar a cada arista una cantidad fija de forma de dejar todos los costos positivos y buscar el camino más corto ¿por qué esto no funciona? Busque un contraejemplo donde esto no funcione.
5. Un colegio debe reparar los pizarrones de sus salas regularmente, por lo cual está planificando esta reparación para los siguientes 7 años (2011 - 2017). La dirección del colegio ha indicado que cada pizarrón debe ser reparado después de 2 años de uso y antes de los 5 años de uso. El costo de esta reparación depende de cuantos años de uso tiene el pizarrón. La siguiente tabla muestra el costo de reparar un pizarrón, dependiendo de cuándo fue comprado y cuántos años de uso tiene:

	2 años uso	3 años uso	4 años uso
2011	3.800	4.100	6.800
2012	4.000	4.800	7.000
2013	4.200	5.100	7.200
2014	4.800	5.700	-
2015	5.700	-	-

Resuelva el problema usando Dijkstra. Es decir, construya un grafo apropiado, de forma que el camino más corto corresponda a la estrategia óptima de reparaciones que debe realizar el colegio.